



ATTENDANCE MANAGER

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF
THE DEGREE OF

B.TECH. (COMPUTER ENGINEERING)

TO

RK UNIVERSITY, RAJKOT

SUBMITTED BY

Name of Student

Shiyal Rutvik

Enrollment No.

24SOEIT13019

UNDER THE GUIDANCE OF

Internal Guide

Prof. Jay Pithadiya

Assistant Professor

RK University

Rajkot

November 2026



SCHOOL OF ENGINEERING, RK UNIVERSITY, RAJKOT

CANDIDATE'S DECLARATION

We hereby certify that We are the sole author(s) of this project work and that neither any part of this project work nor the whole of the project work has been submitted for a degree to any other University or Institution. We certify that, to the best of our knowledge, our project work does not infringe upon anyone's copyright nor violate any proprietary rights and that any ideas, techniques, quotations, or any other material from the work of other people included in our project document, published or otherwise, are fully acknowledged in accordance with the standard referencing practices. We declare that this is a true copy of our project work, including any final revisions, as approved by our project review committee.

Signature of Student (S): _____

Rutvik Shiyal (24SOEIT13019)

Date: _____

Place: _____



CERTIFICATE

This is to certify that the work which is being presented in the Project Report entitled “Attendance Manager”, in partial fulfillment of the requirement for the award of the degree of B.Tech. (Computer Engineering) submitted to the School of Engineering, RK University, is an authentic record of our own work carried out during a period from April 2026 to December 2026

The matter presented in this Project Report has not been submitted by us for the award of any other degree elsewhere.

Signature of Student (S):

Rutvik Shiyal (24SOEIT13019)

This is to certify that the above statement made by the student(s) is correct to the best of my knowledge.

Internal Guide

Prof. Jay Pithadiya
Assistant Professor,
RK University,
Rajkot

Head of Department

Dr. Chetan Shingadiya
CE / IT,
School of Engineering,
RK University, Rajkot



SCHOOL OF ENGINEERING, RK UNIVERSITY, RAJKOT

ACKNOWLEDGEMENT

It is with deep gratitude and respect that I express my sincere appreciation to everyone who has contributed to the successful completion of this project. Their support, guidance, and encouragement have been invaluable throughout this journey.

I would like to extend my deepest gratitude to my internal guide, Prof. Jay Pithadiya, Assistant Professor, School of Engineering, RK University, for her constant guidance, constructive feedback, and intellectual support during the research and development phases of this application. His insights into component layout and logical workflows greatly improved the software design.

I am also highly indebted to Dr. Chetan Shingadiya, Head of Department (CE/IT), School of Engineering, for providing conducive academic conditions and state-of-the-art facilities to pursue practical mobile software engineering.

Finally, I would like to express my appreciation to my peers and family, whose patience, understanding, and encouragement enabled me to balance academic timelines and software deliverables. This project is a result of continuous learning and collective guidance, standing on the shoulders of the mentors who paved the way.

Rutvik Shiyal

Enrollment No: 24SOEIT13019

ABSTRACT

Software Quality Assurance (QA) and test case design represent critical yet labor-intensive stages in the Software Development Life Cycle (SDLC). Traditional manual test writing suffers from high human oversight, inconsistent test coverage, slow authoring rates, and missed boundary and edge cases.

To overcome these operational bottlenecks, this project introduces the AI-Powered Test Case Generator, an intelligent web application designed to automatically convert raw software specifications (User Stories, Feature Descriptions, and API Endpoint Specs) into comprehensive test case suites.

Developed utilizing a modern full-stack architecture, comprising a React + Vite + Tailwind CSS frontend and a Node.js + Express RESTful backend integrated with Large Language Models (LLMs), the application generates structured positive, negative, and boundary test scenarios instantly.

Key architectural features include:

- ❖ Multi-Format Input Parser: support for User Stories, business logic feature descriptions, and API specifications.
- ❖ Contextual Test Generation Engine: automatic identification of acceptance criteria, happy path flows, negative error paths, and numeric/string boundary conditions.
- ❖ Advanced Interactive Data Table: real-time keyword search, priority filtering (High/Medium/Low), type filtering (Positive/Negative/Boundary), multi-row selection, and expandable execution steps.
- ❖ Multi-Format Export Subsystem: native export to Microsoft Excel (.xlsx via SheetJS), comma-separated values (.csv), and plain-text clipboard formatting.
- ❖ Responsive Dashboard & Theme Support: analytics stat cards, toast alert system, and full dark-mode UI support.

The result is a responsive, highly efficient QA utility that reduces test authoring effort significantly while standardizing test documentation across software engineering teams.

ABBREVIATIONS AND NOTATIONS

The following table lists the definitions of abbreviations, terminology, and naming conventions used throughout this project report:

Abbreviation / Term	Definition
QA	Quality Assurance - Software testing and verification process
SDLC	Software Development Life Cycle
LLM	Large Language Model (Artificial Intelligence model)
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation - Data interchange format
UI / UX	User Interface / User Experience
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, and Delete operations
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
CSV	Comma-Separated Values

LIST OF FIGURES

Figure No. / Title	Page No.
Figure 4.1: Use Case Diagram	<i>pg.</i>
Figure 4.2: E-R Diagram	<i>pg.</i>
Figure 4.3: Class Diagram	<i>pg.</i>
Figure 4.4: Data Flow Diagram (Level 0)	<i>pg.</i>
Figure 4.5: Data Flow Diagram (Level 1)	<i>pg.</i>

LIST OF TABLES

Table No. / Title	Page No.
Table 2.1: Project Sprint Schedule	<i>pg.</i>
Table 4.1: Test Suite Entity Schema	<i>pg.</i>
Table 4.2: Test Case Entity Schema	<i>pg.</i>
Table 4.3: User Preference & Session Schema	<i>pg.</i>

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION	9
1.1 Project Summary.....	9
1.2 Purpose: Goals & Objectives.....	9
1.3 Scope (Capabilities & Limitations)	9
Application Capabilities.....	9
Limitations.....	10
1.4 Technology and Literature Review	10
CHAPTER 2: PROJECT MANAGEMENT.....	11
2.1 Project Planning and Scheduling.....	11
2.1.1 Project Development Approach (Process Paradigm) and Justification	11
2.1.2 Project Plan (Milestones, Deliverables, Roles, Responsibilities, and Dependencies).....	11
2.1.3 Schedule Representation	11
2.2 Risk Management	11
2.2.1 Risk Identification.....	11
2.2.2 Risk Analysis	12
2.2.3 Risk Planning	12
CHAPTER 3: SYSTEM REQUIREMENTS STUDY	13
3.1 User Characteristics	13
3.2 Hardware and Software Requirements	13
Development Environment Requirements	13
Client System Minimum Requirements	13
3.3 Constraints	13
CHAPTER 4: SYSTEM ANALYSIS	14
4.1 Study of Current System	14
4.2 Problem and Weaknesses of Current System.....	14
4.3 Requirements of Proposed System.....	14
Functional Requirements.....	14
Non-Functional Requirements.....	14
4.4 Feasibility Study	14
4.5 Requirements Validation	Error! Bookmark not defined.
4.6 Functions of System	14
4.6.1 Use Cases, Event Trace or Scenario (Use Case Diagram)	15
4.7 Data Modeling.....	15
4.7.1 Data Dictionary	15

4.7.2 E-R Diagram.....	16
4.7.3 Class Diagram.....	17
4.7.4 System Activity.....	18
4.7.5 Object Interaction Diagram.....	18
4.8 Functional and Behavioral Modeling	18
4.8.1 Data Flow Diagram (DFD).....	18
4.8.2 Process Specification and Decision Table	19
4.8.3 Control Flow Diagram	19
4.9 Main Modules of New System	19
4.10 Selection of Hardware and Software and Justification.....	19

CHAPTER 1: INTRODUCTION

1.1 Project Summary

The 'AI-Powered Test Case Generator' is a full-stack web application engineered to automate the creation of software Quality Assurance (QA) test cases using Artificial Intelligence. In modern software development, writing comprehensive test cases for user stories, requirements, and REST APIs requires substantial time and domain expertise. Manual test creation often leads to missing edge cases, inconsistent step formatting, and delayed release cycles.

The proposed application resolves these inefficiencies by offering a centralized web workspace where developers and QA engineers can input raw requirement text, select the input specification format (User Story, Feature Description, or API Endpoint Spec), and generate structured test cases instantly.

The application categorizes generated test cases into Positive (Happy Path), Negative (Error Path), and Boundary (Input Limit) scenarios, complete with test case IDs, preconditions/steps, test data, expected results, and priority rankings.

1.2 Purpose: Goals & Objectives

The primary goal of the AI-Powered Test Case Generator is to replace slow manual test creation with an automated, AI-driven workflow. Specific objectives include:

- ❖ **Automated Test Generation:** convert unstructured requirement text into standardized test case tables in seconds.
- ❖ **Comprehensive Scenario Coverage:** ensure every generated test suite automatically includes positive validation, negative error handling, and boundary value conditions.
- ❖ **Multi-Input Support:** tailor prompt templates specifically for User Stories (UI acceptance criteria), Business Features, and REST API Endpoints (HTTP status codes, headers, payloads).
- ❖ **Real Export Capabilities:** provide native exports to Microsoft Excel (.xlsx), Comma-Separated Values (.csv), and plain-text clipboard format for direct integration into QA tools (such as Jira or TestRail).
- ❖ **Interactive UI & Filtering:** enable instant keyword search, priority filtering, type filtering, expandable execution steps, and inline row management.

1.3 Scope (Capabilities & Limitations)

Application Capabilities

- ❖ **Specification Type Parsing:** select between User Story, Feature Description, and API Endpoint modes.
- ❖ **Live Character Validation:** input textarea equipped with real-time character counters (120/1000) and limit warnings.
- ❖ **Dynamic Table Management:** multi-select row selection, "Select All" toggle, inline row deletion, and expandable multi-step lists.
- ❖ **Multi-Format Export Engine:** export full or selected rows to Excel (.xlsx), CSV, and formatted plain text.
- ❖ **Modern User Experience:** floating toast alerts, responsive card layout, dashboard analytics, and full dark-mode toggle.

Limitations

- ❖ Client/Server Dependency: requires a browser environment and connectivity to the Node.js/Express backend server.
- ❖ AI Rate Limits: dependent on API quota limits when connected to the Google Gemini API.

1.4 Technology and Literature Review

Traditional QA test authoring relies on manual spreadsheet entry or enterprise test management portals (e.g., Jira, TestRail, Zephyr). While enterprise portals centralize test artifacts, they do not assist in generating test content, requiring testers to manually write every step, precondition, and expected result.

Recent advancements in Large Language Models (LLMs), including the Google Gemini API used in this project, have demonstrated high proficiency in natural language processing and structured data generation. By combining React (Virtual DOM rendering), Tailwind CSS (utility-first styling), Express.js (REST API layer), and LLM prompt engineering, this project bridges the gap between raw requirements and actionable QA test suites.

CHAPTER 2: PROJECT MANAGEMENT

2.1 Project Planning and Scheduling

2.1.1 Project Development Approach (Process Paradigm) and Justification

An Agile, component-based development paradigm was selected for this project. Given the iterative nature of web applications and UI refinement, Agile allowed modular implementation across weekly sprints, focusing sequentially on API endpoints, state management, interactive table components, export handlers, and dark-mode styling.

2.1.2 Project Plan (Milestones, Deliverables, Roles, Responsibilities, and Dependencies)

Deliverables for the project include:

- ❖ Sprint 1 (Backend & Schema): Express server setup, CORS configuration, route handlers, and structured test case JSON schema.
- ❖ Sprint 2 (Core UI & Input Form): React frontend scaffold, requirement input form, character counter validation, and loading spinner status animation.
- ❖ Sprint 3 (Table & Filtering): data table component, priority badges, type tags, keyword search, priority/type dropdown filters, row checkboxes, and step expander.
- ❖ Sprint 4 (Exports & Polish): SheetJS (xlsx) Excel export, CSV download, clipboard copy, toast alert system, dashboard stat cards, and dark-mode toggle.

Roles and Responsibilities: *[Add team member names and responsibilities here, if this is a group project]*

2.1.3 Schedule Representation

Table 2.1: Project Sprint Schedule

Sprint	Major Milestone / Task	Duration	Status
Sprint 1	Express server setup, route endpoints, CORS & mock JSON controller	Weeks 1-2	Completed
Sprint 2	React Vite scaffold, Tailwind styling, input form & loading status spinner	Weeks 3-4	Completed
Sprint 3	Test case data table, search box, Priority/Type filters, expandable steps	Weeks 5-6	Completed
Sprint 4	SheetJS .xlsx Excel export, CSV download, toast alerts & dark mode	Weeks 7-8	Completed

2.2 Risk Management

2.2.1 Risk Identification

- ❖ API Rate Limiting / Latency: delays or failure during external AI API response generation.
- ❖ Data Export Formatting Incompatibility: inconsistent column rendering across Microsoft Excel versions.
- ❖ UI Performance Degradation: lag when rendering large lists of test case rows.

2.2.2 Risk Analysis

- ❖ API Rate Limiting / Latency: Probability: Medium, Impact: High.
- ❖ Data Export Formatting Incompatibility: Probability: Low, Impact: Medium.
- ❖ UI Performance Degradation: Probability: Medium, Impact: Medium.

2.2.3 Risk Planning

- ❖ Latency Mitigation: implemented an animated step-by-step progress spinner ("Analyzing requirement...", "Calculating boundary limits...") and fallback mock data handlers to keep the application responsive even if backend connectivity drops.
- ❖ Export Mitigation: utilized standard SheetJS xlsx binary array utilities to generate native OpenXML .xlsx spreadsheets compatible with Microsoft Excel, Google Sheets, and LibreOffice.
- ❖ UI Performance Mitigation: applied key-based React list virtualization and efficient state filtering.

CHAPTER 3: SYSTEM REQUIREMENTS

STUDY

3.1 User Characteristics

- ❖ QA Engineers & Testers: require rapid creation of test suites with export options to standard test management tools.
- ❖ Software Developers: need quick unit/integration test scenario outlines before writing code (TDD approach).
- ❖ Product Managers / Business Analysts: verify acceptance criteria coverage for user stories before sprint planning.

3.2 Hardware and Software Requirements

Development Environment Requirements

- ❖ OS: Windows 10/11, macOS, or Linux
- ❖ RAM: 8 GB Minimum
- ❖ IDE: VS Code (Visual Studio Code)
- ❖ Runtime: Node.js (v18.0.0 or higher) and npm (v9.0.0 or higher)

Client System Minimum Requirements

- ❖ Browser: modern web browser (Google Chrome 90+, Mozilla Firefox 88+, Microsoft Edge 90+)
- ❖ Display: responsive resolution (320px mobile to 1920px desktop)
- ❖ RAM: 4 GB RAM

3.3 Constraints

- ❖ Network Protocol: HTTP/HTTPS communication between frontend (Vite dev server / port 5173) and backend (Express server / port 5000).
- ❖ Format Schema: strict adherence to JSON output structure (ID, Scenario, Steps, Test Data, Expected Result, Priority, Type) to ensure seamless table rendering.
- ❖ External Dependency: reliant on the Google Gemini API for test-case generation; unavailability of the API limits core functionality.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Study of Current System

Currently, software development teams generate test cases using manual spreadsheets (Excel/Google Sheets) or text documents. QA engineers read user stories and manually type out every test step, precondition, and expected outcome.

4.2 Problem and Weaknesses of Current System

- ❖ High Human Overhead: takes 20-45 minutes to author a comprehensive test suite for a single feature.
- ❖ Inconsistent Edge Case Coverage: testers frequently forget boundary values or negative invalid input cases.
- ❖ No Standardization: different team members format test steps inconsistently.

4.3 Requirements of Proposed System

Functional Requirements

- ❖ Allow selection of requirement input types (User Story, Feature Description, API Endpoint Spec).
- ❖ Provide input validation with real-time character limit enforcement (1000 characters max).
- ❖ Generate categorized test cases with distinct Priority (High, Medium, Low) and Type (Positive, Negative, Boundary).
- ❖ Support instant keyword searching across test scenarios, steps, and expected results.
- ❖ Provide filter controls for Priority and Type attributes.
- ❖ Enable row selection checkboxes and bulk action handlers.
- ❖ Perform real file downloads for Microsoft Excel (.xlsx) and Comma-Separated Values (.csv).
- ❖ Copy plain-text test suites directly to the clipboard.

Non-Functional Requirements

- ❖ Usability: clean blue/indigo QA color scheme, dark mode toggle, and floating toast notifications.
- ❖ Performance: response rendering in under 1.5 seconds.
- ❖ Reliability: graceful fallback error UI and server offline handling.

4.4 Feasibility Study

- ❖ Technical Feasibility: High. React 18, Express, Tailwind CSS, and SheetJS are production-proven technologies.
- ❖ Operational Feasibility: High. The web layout requires zero training for QA personnel.
- ❖ Economic Feasibility: High. Built using open-source libraries, a standard Node.js runtime, and the Google Gemini API.

4.6 Functions of System

4.6.1 Use Cases, Event Trace or Scenario (Use Case Diagram)

The system's primary use case involves the User (Developer / QA Engineer) interacting with the Input Module to submit a requirement, triggering the AI Generation Engine to produce a structured test suite, which is then filtered, reviewed, and exported.

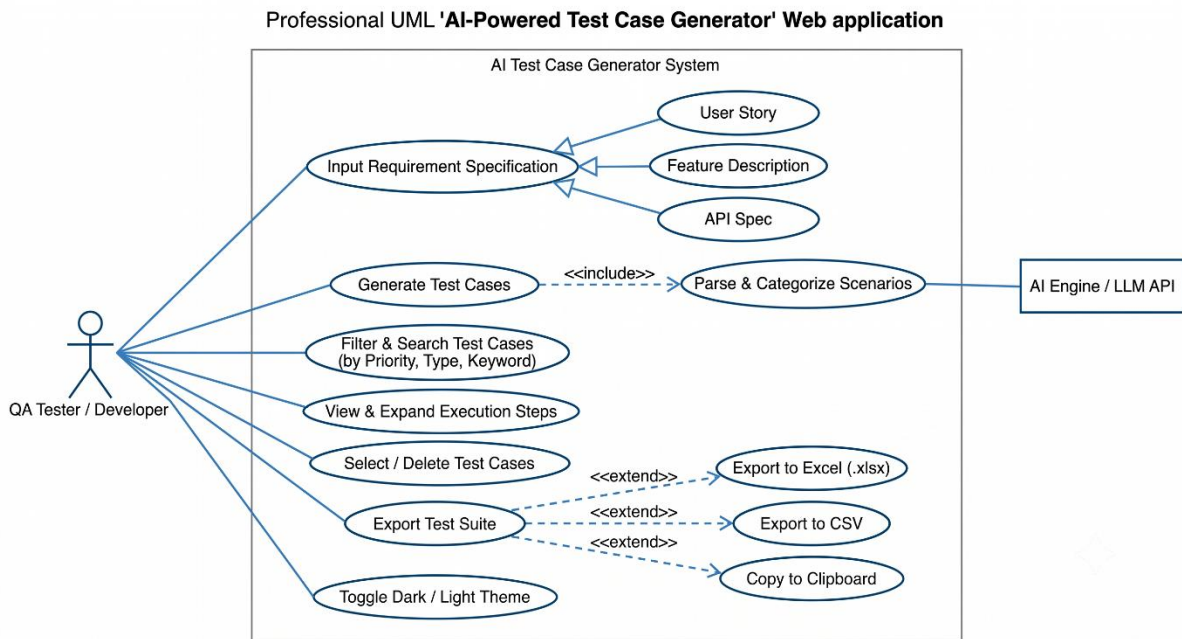


Figure 4.1: Use Case Diagram

4.7 Data Modeling

4.7.1 Data Dictionary

The Data Dictionary provides a specification of the entities, attributes, and relationships used by the AI-Powered Test Case Generator.

Table 4.1: Test Suite Entity Schema

Field	Data Type	Description
id	String	Unique identifier for the generated test suite.
title	String	Title or name of the requirement/feature under test.
inputType	String	Specification type: User Story, Feature Description, or API Endpoint.
createdAt	Timestamp	Date and time the test suite was generated.
testCases	Array	Collection of individual Test Case records belonging to this suite.

Table 4.2: Test Case Entity Schema

Field	Data Type	Description
id	String	Unique identifier for the test case.
scenario	String	Short description of the scenario being tested.
steps	Array of Strings	Ordered list of execution steps.
testData	String	Sample input data used to execute the test.
expectedResult	String	The expected system outcome.
priority	String	High / Medium / Low.
type	String	Positive / Negative / Boundary.

Table 4.3: User Preference & Session Schema

Field	Data Type	Description
theme	<i>e.g. String</i>	<i>light / dark mode preference</i>
lastInputType	<i>e.g. String</i>	<i>most recently used specification type</i>
sessionId	<i>e.g. String</i>	<i>identifier for the current browser session</i>

4.7.2 E-R Diagram

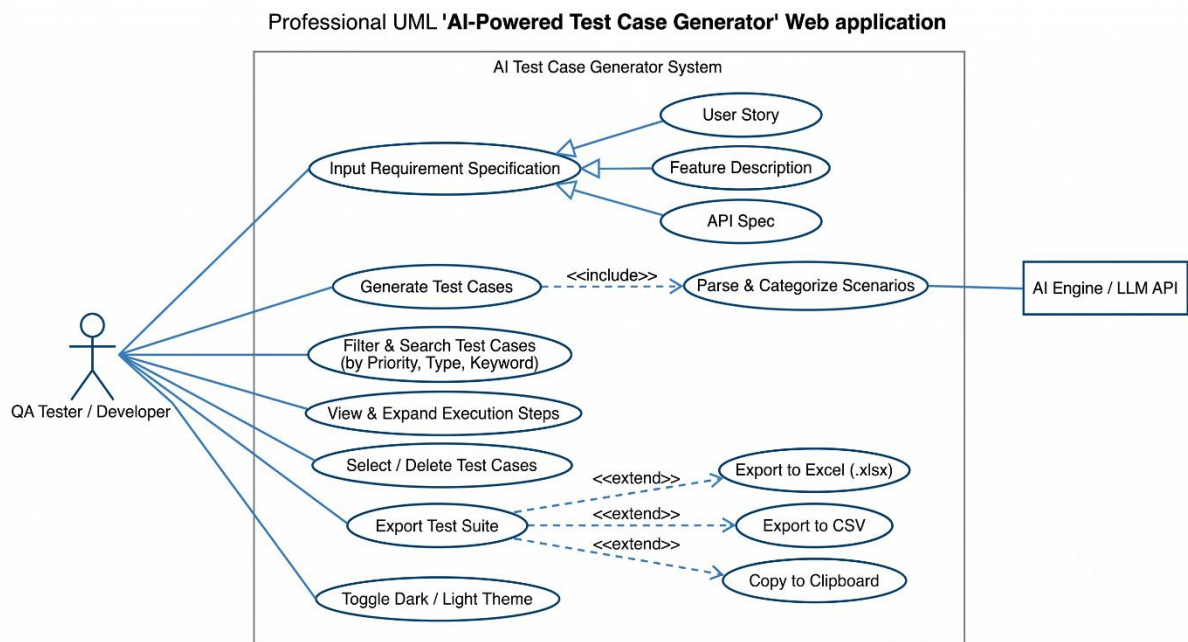


Figure 4.2: E-R Diagram

4.7.3 Class Diagram

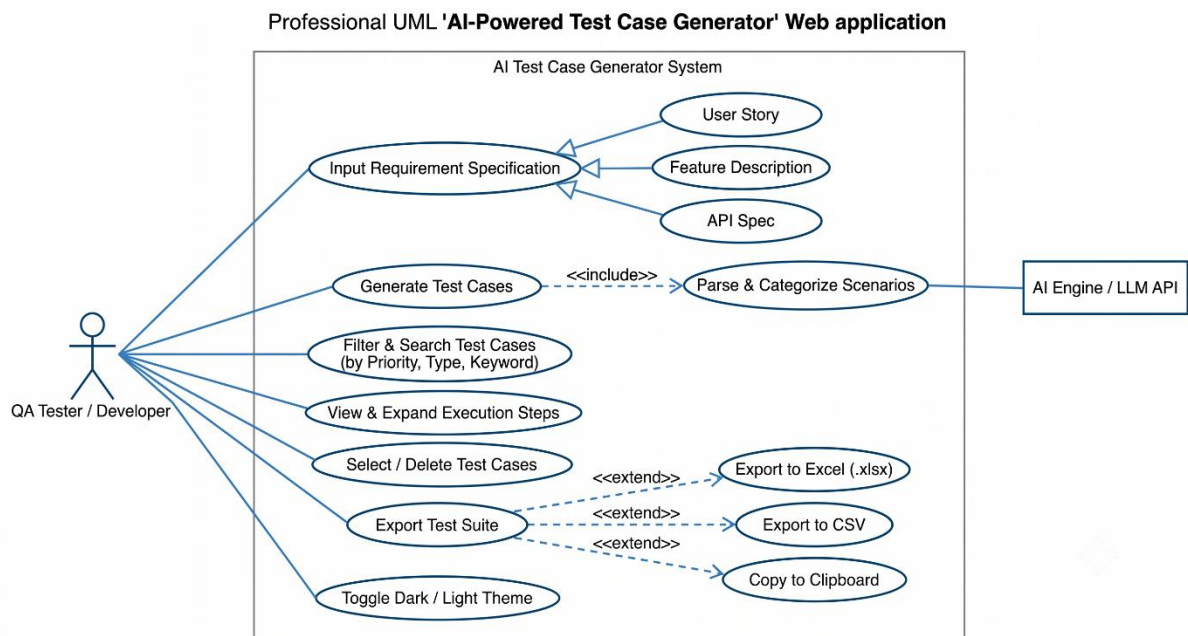


Figure 4.3: Class Diagram

4.7.4 System Activity

Users navigate: Home / Input Form -> Select Input Type -> Submit Requirement -> Review Generated Test Suite -> Filter/Search -> Export.

4.7.5 Object Interaction Diagram

User interactions trigger the API Handler, which calls the LLM service, formats the JSON response, and updates the React component state to re-render the data table.

4.8 Functional and Behavioral Modeling

4.8.1 Data Flow Diagram (DFD)

- ❖ Level 0: the external entity (User) inputs a requirement into the System. The System outputs a rendered test-case table and export files.
- ❖ Level 1: data flows from the Input Parser -> AI Generator / API Handler -> JSON Formatter -> Table Render Engine and Export Subsystem.

AI-Powered Test Case Generator Context DFD (Level 0)

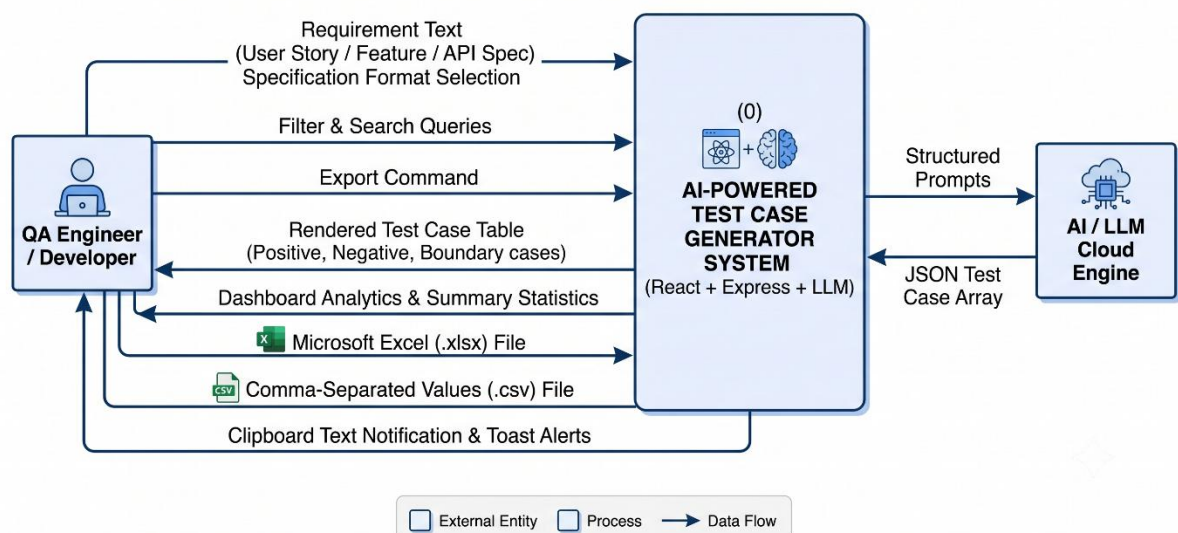


Figure 4.4: Data Flow Diagram (Level 0)

Level 1 Data Flow Diagram for AI-Powered Test Case Generator

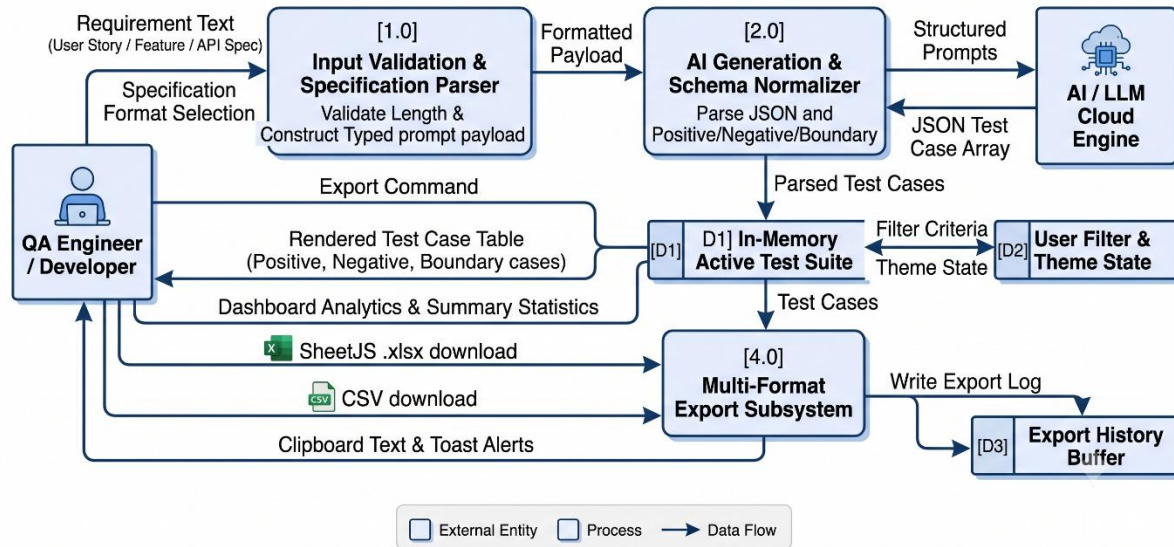


Figure 4.5: Data Flow Diagram (Level 1)

4.8.2 Process Specification and Decision Table

If 'Character count > 1000', block submission; if 'API response fails', display fallback/mock data; if 'Export clicked', generate the selected file format (.xlsx / .csv / clipboard text).

4.8.3 Control Flow Diagram

Landing / Input Form -> Loading State -> Results Dashboard -> [Search / Filter / Export] -> Done.

4.9 Main Modules of New System

- ❖ Input Module: manages requirement text entry, input-type selection, and character validation.
- ❖ Generation Module: handles communication with the Google Gemini API and structures the AI response.
- ❖ Table & Filtering Module: renders the interactive test-case table with search, filters, and row selection.
- ❖ Export Module: manages Excel (.xlsx), CSV, and clipboard export logic.
- ❖ UI/Theme Module: manages dashboard analytics, toast notifications, and dark-mode styling.

4.10 Selection of Hardware and Software and Justification

- ❖ React (Vite): chosen for fast component-based rendering and quick development builds.
- ❖ Tailwind CSS: chosen for rapid, utility-first responsive styling and consistent design tokens.
- ❖ Node.js + Express: chosen for a lightweight, well-supported REST API layer.
- ❖ Google Gemini API: chosen for its natural-language understanding and structured JSON generation capability.
- ❖ SheetJS (xlsx): chosen for reliable, spreadsheet-application-compatible Excel export.